

Hackathon Project Submission – Database Administration

College Course Registration Database

Student Details

Student Name	Agalya M
Register Number	2422K2135
Department	Bsc Computer Science
ID	asbru4w2422k2135
College	Government Arts and Science College - Gudalur
Date of Submission	22-09-2026

1. Problem Statement & Objectives

Problem Statement

In an educational institution, manually handling course registrations, allocating faculty members to courses, and tracking enrolled students leads to data redundancy, scheduling conflicts, and human errors. The objective of this project is to design and implement a centralized relational database system ('College Course Registration Database') to efficiently manage student records, course details, faculty assignments, and course enrollments, enabling error-free course-wise student tracking

Objectives

- To store and maintain records of students, faculty, and academic courses.
- To implement a junction table structure that handles many-to-many relationships between students and courses without duplication.
- To assign faculty members to specific courses using foreign key relationships.
- To generate course-wise student enrollment reports seamlessly using SQL JOIN queries.

2. Database Design

2.1 Database Structure

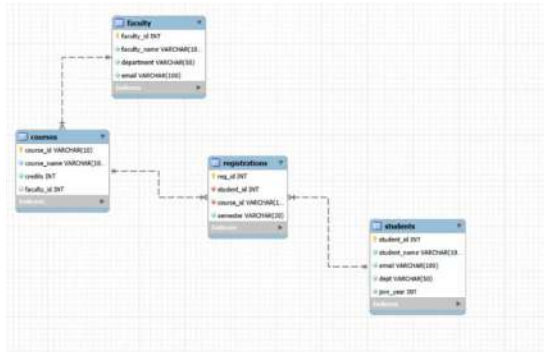
The **College Course Registration Database** is designed to manage academic course allocations, student enrollments, and faculty assignments. It contains four normalized relational tables that eliminate data redundancy and handle many-to-many relationships between students and courses through a structured junction table.

Tables Used

Table Name	Purpose
Faculty	Stores details of teaching faculty members including their ID, name, department, and contact email.
Students	Stores student details including student ID, name, email, department, and year of joining.
Courses	Stores information about courses offered and links each course to the assigned faculty member.
Registrations	Acts as a junction table to manage student enrollments in courses across semesters.

2.2 Entity Relationship Diagram

Diagram:



Entities and Attributes:

- **Faculty:** faculty_id (Primary Key), faculty_name, department, email
- **Students:** student_id (Primary Key), student_name, email, dept, join_year
- **Courses:** course_id (Primary Key), course_name, credits, faculty_id (Foreign Key referencing Faculty)
- **Registrations:** reg_id (Primary Key), student_id (Foreign Key referencing Students), course_id (Foreign Key referencing Courses), semester

2.3 Technologies Used

Technology / Tool	Purpose
MySQL	Database management system
MySQL Workbench	Database design and SQL execution
SQL	Database creation, manipulation, and querying

3. Practical Implementation

3.1 Database and Table Creation

-- Database Creation

CREATE DATABASE CollegeDB;

USE CollegeDB;

-- 1. Faculty Table

```
CREATE TABLE Faculty (  
  
    faculty_id INT PRIMARY KEY AUTO_INCREMENT,  
  
    faculty_name VARCHAR(100) NOT NULL,  
  
    department VARCHAR(50) NOT NULL,  
  
    email VARCHAR(100) UNIQUE NOT NULL  
  
);
```

-- 2. Students Table

```
CREATE TABLE Students (  
  
    student_id INT PRIMARY KEY AUTO_INCREMENT,  
  
    student_name VARCHAR(100) NOT NULL,  
  
    email VARCHAR(100) UNIQUE NOT NULL,  
  
    dept VARCHAR(50) NOT NULL,  
  
    join_year INT NOT NULL  
  
);
```

-- 3. Courses Table

CREATE TABLE Courses (

course_id VARCHAR(10) PRIMARY KEY,

course_name VARCHAR(100) NOT NULL,

credits INT NOT NULL,

faculty_id INT,

FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)

);

-- 4. Registrations Table (Junction Table)

CREATE TABLE Registrations (

reg_id INT PRIMARY KEY AUTO_INCREMENT,

student_id INT NOT NULL,

course_id VARCHAR(10) NOT NULL,

semester VARCHAR(20) NOT NULL,

FOREIGN KEY (student_id) REFERENCES Students(student_id),

FOREIGN KEY (course_id) REFERENCES Courses(course_id),

UNIQUE (student_id, course_id, semester)

);

Database Creation

```
CREATE DATABASE CollegeDB;  
USE CollegeDB;
```

Customer Table

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY AUTO_INCREMENT,  
    student_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    dept VARCHAR(50) NOT NULL,  
    join_year INT NOT NULL  
);
```

STUDEMETS Table

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY AUTO_INCREMENT,  
    student_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    dept VARCHAR(50) NOT NULL,  
    join_year INT NOT NULL  
);
```

Service Table

```
CREATE TABLE Courses (  
    course_id VARCHAR(10) PRIMARY KEY,
```

```

course_name VARCHAR(100) NOT NULL,
credits INT NOT NULL,
faculty_id INT,
FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);

```

3.2 Constraints and Relationships

Explain the primary keys, foreign keys, and other constraints applied to the tables.

Table	Primary Key	Foreign Key	Other Constraints
Faculty	faculty_id	None	NOT NULL on name/dept, UNIQUE on email, AUTO_INCREMENT
Students	student_id	None	NOT NULL on name/dept/year, UNIQUE on email, AUTO_INCREMENT
Courses	course_id	faculty_id (references Faculty)	NOT NULL on name/credits
Registrations	reg_id	faculty_id (references Faculty)	NOT NULL on semester, AUTO_INCREMENT, UNIQUE (student_id, course_id, semester)

3.3 Customer Data

Faculty Data

```
INSERT INTO Faculty (faculty_name, department, email) VALUES  
(  
'Dr. Ramesh Kumar', 'Computer Science', 'ramesh@college.edu'),  
(  
'Prof. Priya Sharma', 'Mathematics', 'priya@college.edu');
```

Students data

```
INSERT INTO Students (student_name, email, dept, join_year) VALUES  
(  
'Deepak', 'deepak@student.edu', 'B.Sc CS', 2024),  
(  
'Sanjay', 'sanjay@student.edu', 'B.Sc CS', 2024),  
(  
'Kavitha', 'kavitha@student.edu', 'B.Sc Maths', 2024);
```

Courses Data

```
INSERT INTO Courses (course_id, course_name, credits, faculty_id) VALUES  
(  
'CS201', 'Database Management Systems', 4, 1),  
(  
'MA201', 'Discrete Mathematics', 3, 2);
```

Registration Data

```
INSERT INTO Registrations (student_id, course_id, semester) VALUES  
(  
1, 'CS201', 'Semester 3'),  
(  
2, 'CS201', 'Semester 3'),  
(  
1, 'MA201', 'Semester 3'),  
(  
3, 'MA201', 'Semester 3');
```

3.4 SQL Operations

INSERT

-- Inserting a new student record

```
INSERT INTO Students (student_name, email, dept, join_year)
VALUES ('Arun Kumar', 'arun@student.edu', 'B.Sc CS', 2024);
```

-- Enrolling the new student into a course

```
INSERT INTO Registrations (student_id, course_id, semester)
VALUES (4, 'CS201', 'Semester 3');
```

SELECT

-- Retrieve course-wise enrolled students with instructor details

```
SELECT
    c.course_id AS 'Course Code',
    c.course_name AS 'Course Name',
    f.faculty_name AS 'Faculty Name',
    s.student_id AS 'Student ID',
    s.student_name AS 'Student Name',
    r.semester AS 'Semester'
FROM Registrations r
JOIN Courses c ON r.course_id = c.course_id
LEFT JOIN Faculty f ON c.faculty_id = f.faculty_id
JOIN Students s ON r.student_id = s.student_id
ORDER BY c.course_id, s.student_name;
```

UPDATE

-- Updating the assigned faculty member for a course

UPDATE Courses

SET faculty_id = 2

WHERE course_id = 'CS201';

-- Updating a student's department or contact email

UPDATE Students

SET email = 'deepak.cs@student.edu'

WHERE student_id = 1;

DELETE

-- Removing a student course registration

DELETE FROM Registrations

WHERE student_id = 4 AND course_id = 'CS201';

WHERE

-- Filter students registered specifically for the course 'CS201'

SELECT student_id, course_id, semester

FROM Registrations

WHERE course_id = 'CS201';

ORDER BY

-- Retrieve all students sorted alphabetically by their name

SELECT student_id, student_name, email, dept

FROM Students

ORDER BY student_name ASC;

LIKE

-- Retrieve all students sorted alphabetically by their name

```
SELECT student_id, student_name, email, dept  
FROM Students  
ORDER BY student_name ASC;
```

BETWEEN

-- Retrieve courses having credits between 3 and 5

```
SELECT course_id, course_name, credits  
FROM Courses  
WHERE credits BETWEEN 3 AND 5;
```

Aggregate Functions

-- Total students, total credits, average credits, max & min credits

```
SELECT  
    COUNT(student_id) AS 'Total Students Enrolled',  
    SUM(credits) AS 'Total Course Credits',  
    AVG(credits) AS 'Average Course Credits',  
    MAX(credits) AS 'Maximum Credits',  
    MIN(credits) AS 'Minimum Credits'  
FROM Courses c  
JOIN Registrations r ON c.course_id = r.course_id;
```

GROUP BY

-- Count number of students registered per course

SELECT

course_id AS 'Course Code',

COUNT(student_id) AS 'Enrolled Students'

FROM Registrations

GROUP BY course_id;

INNER JOIN

-- Retrieve enrolled students with their registered courses using INNER JOIN

SELECT

s.student_id AS 'Student ID',

s.student_name AS 'Student Name',

c.course_id AS 'Course Code',

c.course_name AS 'Course Name',

r.semester AS 'Semester'

FROM Registrations r

INNER JOIN Students s ON r.student_id = s.student_id

INNER JOIN Courses c ON r.course_id = c.course_id;

3.5 Project Execution

The project was executed using **MySQL Workbench** connected to a local MySQL Server instance:

1. **Database and Table Initialization:** A dedicated database schema named CollegeDB was created and selected using CREATE DATABASE and USE statements. The relational tables—Faculty, Students, Courses, and Registrations—were defined with proper primary keys, foreign keys, and unique constraints to maintain referential integrity and prevent duplicate course enrollments.
1. **Execution Process:** The DDL and DML scripts were written directly in the MySQL Workbench SQL query editor tab. The scripts were executed using the execute toolbar button (⚡ **Lightning Bolt**) and verified through the **Action Output** console, confirming error-free execution with green status ticks.
1. **Data Management and Query Testing:** Sample records were inserted into the parent tables (Faculty, Students, Courses) and the junction table (Registrations). Various SQL operations—including SELECT, WHERE, ORDER BY, GROUP BY, and multi-table JOIN queries—were executed to validate the workflow and retrieve the final course-wise student enrollment reports in the interactive **Result Grid**.

4. Output Screenshots

Screenshot 1 – Database and Tables

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 1	16:22:23	CREATE DATABASE CollegeDB	1 row(s) affected	0.000 sec
✓ 2	16:22:23	USE CollegeDB	0 row(s) affected	0.000 sec
✓ 3	16:22:23	CREATE TABLE Faculty (faculty_id INT PRIMARY KEY AUTO_INCREMENT, faculty_name VARCHAR(100) NOT NULL)	0 row(s) affected	0.047 sec
✓ 4	16:22:23	CREATE TABLE Students (student_id INT PRIMARY KEY AUTO_INCREMENT, student_name VARCHAR(100) NOT NULL)	0 row(s) affected	0.031 sec
✓ 5	16:22:23	CREATE TABLE Courses (course_id VARCHAR(10) PRIMARY KEY, course_name VARCHAR(100) NOT NULL)	0 row(s) affected	0.032 sec
✓ 6	16:22:23	CREATE TABLE Registrations (reg_id INT PRIMARY KEY AUTO_INCREMENT, student_id INT NOT NULL, course_id VARCHAR(10) NOT NULL)	0 row(s) affected	0.031 sec
✗ 7	16:22:29	CREATE DATABASE CollegeDB	Error Code: 1007. Can't create database 'collegedb': database exists	0.000 sec
✓ 8	16:27:18	USE CollegeDB	0 row(s) affected	0.000 sec
✓ 9	16:27:18	INSERT INTO Faculty (faculty_name, department, email) VALUES ('Dr. Ramesh Kumar', 'Computer Science', 'ra...)	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.016 sec
✓ 10	16:27:18	INSERT INTO Students (student_name, email, dept, join_year) VALUES ('Deepak', 'deepak@student.edu', 'B.S...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.015 sec
✓ 11	16:27:18	INSERT INTO Courses (course_id, course_name, credits, faculty_id) VALUES ('CS201', 'Database Management...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec
✓ 12	16:27:18	INSERT INTO Registrations (student_id, course_id, semester) VALUES (1, 'CS201', 'Semester 3'), (2, 'CS201', 'S...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec
✓ 13	16:27:48	SELECT c.course_id AS 'Course Code', c.course_name AS 'Course Name', f.faculty_name AS 'Faculty ...	4 row(s) returned	0.000 sec / 0.000 sec

Screenshot 2 – Faculty Table with Records

faculty_id	faculty_name	department	email
1	Dr. Ramesh Kumar	Computer Science	ramesh@college.edu
2	Prof. Priya Sharma	Mathematics	priya@college.edu
NULL	NULL	NULL	NULL

Screenshot 3 – Student Table with Records

student_id	student_name	email	dept	join_year
1	Deepak	deepak@student.edu	B.Sc CS	2024
2	Sanjay	sanjay@student.edu	B.Sc CS	2024
3	Kavitha	kavitha@student.edu	B.Sc Maths	2024
NULL	NULL	NULL	NULL	NULL

Screenshot 4 – Course Table with Records

course_id	course_name	credits	faculty_id
CS201	Database Management Systems	4	1
MA201	Discrete Mathematics	3	2
NULL	NULL	NULL	NULL

Screenshot 5 – Registrations Table with Records

reg_id	student_id	course_id	semester
1	1	CS201	Semester 3
3	1	MA201	Semester 3
2	2	CS201	Semester 3
4	3	MA201	Semester 3
NULL	NULL	NULL	NULL

Screenshot 6 – INSERT / UPDATE / DELETE Operations

22	19:09:40	USE CollegeDB	0 row(s) affected	0.000 sec
23	19:09:40	INSERT INTO Students (student_name, email, dept, join_year) VALUES ('Anun Kumar', 'anun@student.edu', 'B...	1 row(s) affected	0.000 sec
24	19:09:40	UPDATE Students SET email = 'deepak.cs@student.edu' WHERE student_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.016 sec
25	19:09:40	DELETE FROM Registrations WHERE student_id = 4 AND course_id = 'CS201'	0 row(s) affected	0.000 sec

Screenshot 7 – SELECT and Filtering Operations

Course Code	Course Name	Faculty Name	Student ID	Student Name	Semester
CS201	Database Management Systems	Dr. Ramesh Kumar	1	Deepak	Semester 3
CS201	Database Management Systems	Dr. Ramesh Kumar	2	Sanjay	Semester 3
MA201	Discrete Mathematics	Prof. Priya Sharma	1	Deepak	Semester 3
MA201	Discrete Mathematics	Prof. Priya Sharma	3	Kavitha	Semester 3

Screenshot 8 – Aggregate Functions

Total Students Enrolled	Total Course Credits	Average Course Credits	Maximum Credits	Minimum Credits
4	14	3.5000	4	3

Screenshot 9 – GROUP BY Operation

Course Code	Enrolled Students
CS201	2
MA201	2

Screenshot 10 – INNER JOIN Operation

Student ID	Student Name	Course Code	Course Name	Semester
2	Sanjay	CS201	Database Management Systems	Semester 3
1	Deepak	CS201	Database Management Systems	Semester 3
3	Kavitha	MA201	Discrete Mathematics	Semester 3
1	Deepak	MA201	Discrete Mathematics	Semester 3

Screenshot 11 – Final Database / Query Results

Course Code	Course Name	Faculty Name	Student ID	Student Name	Semester
CS201	Database Management Systems	Dr. Ramesh Kumar	1	Deepak	Semester 3
CS201	Database Management Systems	Dr. Ramesh Kumar	2	Sanjay	Semester 3
MA201	Discrete Mathematics	Prof. Priya Sharma	1	Deepak	Semester 3
MA201	Discrete Mathematics	Prof. Priya Sharma	3	Kavitha	Semester 3

5. Technical Explanation

Database Design

Database Design

The database is structured into four distinct normalized tables—Faculty, Students, Courses, and Registrations—to follow the principles of normalization (up to 3NF) and avoid data anomalies.

- **Faculty:** Dedicated to managing instructor profiles and department allocations.
- **Students:** Stores demographic and academic records of learners.
- **Courses:** Holds academic course offerings, credit weights, and links to faculty mentors.
- **Registrations:** Resolves the Many-to-Many relationship between students and courses without repeating full profile details.

By separating the entities into dedicated tables, insertion, update, and deletion anomalies are prevented, and data redundancy is minimized.

Keys and Constraints

- **Primary Keys (PK):** Columns `faculty_id`, `student_id`, `course_id`, and `reg_id` uniquely identify records within their respective tables, preventing duplicate entities.
- **Foreign Keys (FK):** `Courses(faculty_id)` enforces a 1:N link with Faculty, while `Registrations(student_id)` and `Registrations(course_id)` enforce referential integrity with Students and Courses, blocking registrations for nonexistent students or courses.

- **Other Constraints:**

- NOT NULL: Applied on mandatory attributes (names, departments, credits, semesters).
- UNIQUE: Enforced on email IDs and a composite constraint
UNIQUE(student_id, course_id, semester) to ensure a student cannot enroll in the same course twice during the same academic term.
- AUTO_INCREMENT: Automatically assigns unique serial keys for newly added instructors, students, and registration entries.

SQL Operations

- **INSERT:** Populates initial schema master data and dynamically records student enrollments.
- **SELECT:** Retrieves stored records, either entirely or filtered across single/multiple tables using joins for administrative review.
- **UPDATE:** Modifies existing information (such as changing course instructors or student contact information) without re-creating records.
- **DELETE:** Removes specific obsolete entries (such as dropping an active course enrollment) using filtered primary or composite keys.

Data Retrieval

- **WHERE:** Filters datasets based on explicit conditions (e.g., retrieving registrations restricted to a specific course code).
- **ORDER BY:** Organizes retrieved data ascendingly or descendingly (e.g., listing student directories alphabetically).
- **LIKE:** Executes pattern-matching searches across text attributes using wildcards (e.g., searching for course titles containing specific keywords).
- **BETWEEN:** Filters records within a defined numeric or date range (e.g., filtering courses offering credits between 3 and 5).
- **GROUP BY:** Aggregates row subsets by a specific attribute (e.g., calculating total enrollment counts grouped by course).

Aggregate Functions and Joins

- **Aggregate Functions:** Standard functions (COUNT, SUM, AVG, MIN, MAX) summarize transactional metrics across the database, allowing institutional reporting on student counts, total credit load, and credit range distributions.
- **INNER JOIN:** Used to reconstruct relational links between Registrations, Students, and Courses on matching foreign keys, pulling relational datasets together into a single cohesive report displaying student names, enrolled subjects, credit values, and faculty mentors.

6. Conclusion

Conclusion

The "College Course Registration Database" project successfully implements a centralized relational database architecture using MySQL. By normalizing the schema into four distinct tables (Faculty, Students, Courses, and Registrations), the system eliminates data redundancy and maintains strong referential integrity. The junction table effectively handles many-to-many relationships, preventing duplicate enrollments through composite constraints. Core SQL operations—including data insertion, filtering, sorting, aggregate functions, and complex joins—were executed to verify workflow accuracy and generate comprehensive course enrollment reports. Through this project, core database administration concepts such as DDL/DML execution, primary/foreign key relationships, and query optimization were practically applied.

Future Enhancements

- **Web and Mobile Application Interface:** Integrating a front-end portal (using React, HTML/CSS, or mobile apps) to allow students and faculty members to register and view course allocations directly through a user interface.
- **Automated Capacity Limits and Triggers:** Implementing SQL triggers to enforce maximum seat limits per course and automatically handle waitlists when a course is full.
- **Grade and Attendance Tracking:** Expanding the schema with additional entities for tracking attendance, semester examination marks, and GPA calculations.
- **Role-Based Access Control (RBAC):** Configuring granular MySQL user permissions for administrators, faculty, and students to enhance database security.